UNIVERSITÉ ABDELMALEK ESSAADI
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES D'AL-HOCEIMA

Département : Mathématiques et Informatique

Filière : 2ème année Génie Informatique

Module : Machine Learning

Rapport de Projet

Prédiction de la CAN 2025

Système de prédiction du tournoi de la Coupe d'Afrique des Nations utilisant le Machine Learning

Réalisé par : **Mohssine ECHLAIHI**
Hassan EL KHATOURY

Encadré par : **Pr. Anouar RAGRAGUI**

Année Universitaire : 2025-2026

Table des matières

1 Introduction **3**
Contexte . . . 3
Problématique . . . 3
Objectifs du projet . . . 3
Méthodologie . . . 3

2 Rappels théoriques sur le Machine Learning **5**
Qu'est-ce que le Machine Learning ? . . . 5
Random Forest Classifier . . . 5
Métriques d'évaluation . . . 5
Simulation Monte Carlo . . . 6

3 Données utilisées **7**
Sources de données . . . 7
Nettoyage des données (Script 1) . . . 7

4 Ingénierie des caractéristiques **8**
Importance du Feature Engineering . . . 8
Features créées . . . 8
1. Classement FIFA . . . 8
2. Forme récente (5 derniers matchs) . . . 8
3. Confrontations directes (Head-to-Head) . . . 8
4. Palmarès CAN . . . 9
5. Avantage du terrain . . . 9
6. Features composites . . . 9
Dataset final . . . 9

5 Entraînement du modèle **10**
Préparation des données . . . 10
Optimisation des hyperparamètres . . . 10
Sauvegarde des artefacts . . . 11
Architecture du modèle . . . 11

6 Simulation du tournoi **12**
Équipes participantes . . . 12
Fonction de prédiction . . . 12
Simulation Monte Carlo . . . 13

7 Résultats et prédictions **14**
Probabilités de victoire . . . 14

Champion prédit 14
Analyse des résultats 14

8 Application Web **15**
Architecture technique 15
Backend FastAPI 15
Pages de l'application 15
Technologies utilisées 16

9 Structure du projet **17**
Arborescence des fichiers 17
Dépendances 17

10 Conclusion **19**
Récapitulatif 19
Résultat principal 19
Limites et améliorations 19
Compétences acquises 19

Introduction

Contexte

La Coupe d'Afrique des Nations (CAN) est la compétition de football la plus prestigieuse du continent africain. L'édition 2025, organisée au Maroc, rassemble les 16 meilleures équipes nationales africaines.

Dans le domaine du sport, les prédictions basées sur les données sont devenues un outil précieux pour :

— **Les analystes sportifs :** Pour évaluer les forces en présence.

— **Les entraîneurs :** Pour préparer leurs stratégies.

— **Les passionnés :** Pour anticiper les résultats avec des bases scientifiques.

Le Machine Learning offre une approche objective pour analyser l'historique des matchs et prédire les résultats futurs, en tenant compte de multiples facteurs comme le classement FIFA, la forme récente, et les confrontations directes.

Problématique

Ce projet vise à répondre à la question suivante :

> ***Comment utiliser les techniques de Machine Learning pour prédire le vainqueur de la CAN 2025 à partir de données historiques du football africain ?***

Les défis à relever sont :

1. Collecter et nettoyer des données fiables sur les matchs africains.
2. Créer des caractéristiques (features) pertinentes pour la prédiction.
3. Entraîner un modèle capable de prédire victoire, défaite ou match nul.
4. Simuler le tournoi complet pour estimer les probabilités de chaque équipe.

Objectifs du projet

— **Data Engineering :** Nettoyer et préparer 5 jeux de données brutes.

— **Feature Engineering :** Créer 23 caractéristiques pertinentes.

— **Modélisation :** Entraîner un modèle Random Forest optimisé.

— **Simulation :** Utiliser Monte Carlo pour simuler 1000 tournois.

— **Visualisation :** Développer une application web interactive.

Méthodologie

Notre approche suit un pipeline en quatre étapes :

1. **Nettoyage des données** (Script 1)

2. **Ingénierie des caractéristiques** (Script 2)
3. **Entraînement du modèle** (Script 3)
4. **Simulation du tournoi** (Script 4)

Rappels théoriques sur le Machine Learning

Qu'est-ce que le Machine Learning ?

Le Machine Learning (apprentissage automatique) est une branche de l'intelligence artificielle qui permet aux ordinateurs d'apprendre à partir de données sans être explicitement programmés.

Dans notre cas, nous utilisons l'**apprentissage supervisé** : le modèle apprend à partir d'exemples étiquetés (matchs passés avec leurs résultats) pour prédire de nouveaux résultats.

Random Forest Classifier

L'algorithme **Random Forest** (Forêt Aléatoire) est un ensemble d'arbres de décision. Son principe est le suivant :

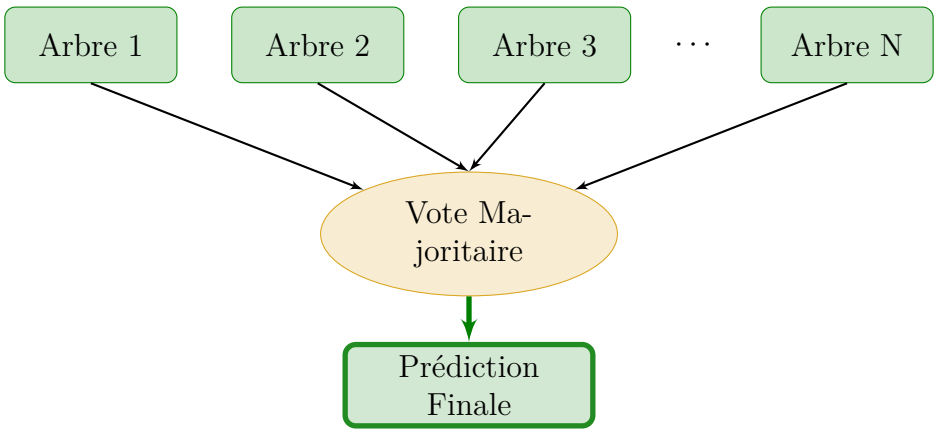


FIGURE 2.1 – Principe du Random Forest : vote majoritaire de plusieurs arbres

Avantages de Random Forest :

- Robuste au surapprentissage (overfitting).
- Gère bien les données hétérogènes.
- Fournit une importance des features.
- Performant sur des datasets de taille moyenne.

Métriques d'évaluation

Pour un problème de classification multi-classes (W, D, L), nous utilisons :

- **Accuracy :** Pourcentage de prédictions correctes.
- **F1-Score Macro :** Moyenne harmonique de précision et rappel, équilibrée entre classes.
- **Matrice de confusion :** Visualisation des erreurs par classe.

Simulation Monte Carlo

La méthode **Monte Carlo** consiste à répéter une simulation aléatoire un grand nombre de fois (1000 dans notre cas) pour estimer des probabilités.

Pour chaque simulation :

1. On prédit le vainqueur de chaque match avec le modèle.
2. On simule tout le tournoi (huitièmes → finale).
3. On enregistre le champion.

Après 1000 simulations, la probabilité de victoire d'une équipe = nombre de victoires / 1000.

Données utilisées

Sources de données

Nous avons collecté 5 jeux de données dans le dossier `raw_data/` :

Fichier	Lignes	Description
African national football 2010-2024.csv	500	Matchs des équipes africaines
Africa Cup of Nations Matches.csv	300	Historique des matchs CAN
fifa_ranking-2024-06-20.csv	211	Classement FIFA mondial
Champions.csv	34	Historique des champions CAN
General Statistics.csv	16	Statistiques par équipe

TABLE 3.1 – Jeux de données sources

Nettoyage des données (Script 1)

Le script `1_data_cleaning.py` effectue les opérations suivantes :

```python
def normalize_team_name(name):
    """Normalise les noms d'equipes: strip, title case"""
    if pd.isna(name):
        return name
    return name.strip().title()

def clean_african_football():
    df = pd.read_csv('raw_data/African national football.csv')

    # Conversion de la date
    df['date'] = pd.to_datetime(df['date'], format='%m/%d/%Y')

    # Normalisation des noms
    df['home_team'] = df['home_team'].apply(normalize_team_name)
    df['away_team'] = df['away_team'].apply(normalize_team_name)

    # Creation de la colonne resultat
    def get_result(row):
        if row['home_score'] > row['away_score']: return 'W'
        elif row['home_score'] < row['away_score']: return 'L'
        else: return 'D'

    df['result'] = df.apply(get_result, axis=1)
    df.to_csv('processed_data/cleaned_african_football.csv')
```

Listing 3.1 – Extrait du nettoyage des données

Opérations effectuées :

— Conversion des dates en format datetime.

— Normalisation des noms d'équipes (majuscules, espaces).

— Suppression des matchs avec données manquantes.

— Création de la colonne résultat (W/D/L).

Ingénierie des caractéristiques

Importance du Feature Engineering

La qualité des caractéristiques (features) détermine en grande partie la performance du modèle. Nous avons créé 23 features regroupées en catégories.

Features créées

1. Classement FIFA

— `fifa_rank_diff` : Différence de classement FIFA entre équipe 1 et 2.

2. Forme récente (5 derniers matchs)

— `team1_last5_points` : Points obtenus sur les 5 derniers matchs (V=3, N=1, D=0).

— `team2_last5_points` : Idem pour l'équipe 2.

— `team1_last5_goal_diff` : Différence de buts moyenne.

— `team2_last5_goal_diff` : Idem pour l'équipe 2.

```python
def calculate_last5_stats(df, team_col, date_col):
    stats = {}
    for idx, row in df.iterrows():
        team = row[team_col]
        match_date = row[date_col]

        # Recuperer les 5 derniers matchs
        past_matches = df[
            ((df['team1'] == team) | (df['team2'] == team)) &
            (df[date_col] < match_date)
        ].tail(5)

        points = 0
        for _, match in past_matches.iterrows():
            if match['team1'] == team:
                points += 3 if match['result'] == 'W' else (1 if match['result'] == 'D' else 0)
            else:
                points += 3 if match['result'] == 'L' else (1 if match['result'] == 'D' else 0)

        stats[idx] = {'points': points}
    return stats
```

Listing 4.1 – Calcul de la forme récente

3. Confrontations directes (Head-to-Head)

— `h2h_total_matches` : Nombre de confrontations passées.

— `h2h_team1_win_rate` : Taux de victoire de l'équipe 1 contre l'équipe 2.
— `h2h_dominance` : Indicateur de domination historique.

4. Palmarès CAN

— `team1_can_titles` : Nombre de titres CAN de l'équipe 1.
— `team2_can_titles` : Idem pour l'équipe 2.
— `titles_advantage` : Différence de titres.

5. Avantage du terrain

— `team1_is_host` : 1 si l'équipe 1 est le pays hôte (Maroc).
— `team2_is_host` : Idem pour l'équipe 2.

6. Features composites

— `form_momentum_diff` : Différence de momentum entre les deux équipes.
— `can_performance_diff` : Différence de performance historique en CAN.

Dataset final

Le fichier `final_dataset_for_modeling.csv` contient :
— **500+ lignes** : matchs historiques.
— **23 colonnes** : features numériques.
— **1 colonne cible** : résultat (W, D, L).

Entraînement du modèle

Préparation des données

Le script `3_train_model.py` prépare les données pour l'entraînement :

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

df = pd.read_csv('processed_data/final_dataset_for_modeling.csv')

# Separation features / cible
X = df.drop(['result', 'match_id', 'date', 'team1', 'team2'], axis=1)
y = df['result']

# Division 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# Normalisation des features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Encodage des labels
label_encoder = LabelEncoder()
y_train_encoded = label_encoder.fit_transform(y_train)
```

Listing 5.1 – Préparation et division des données

Optimisation des hyperparamètres

Nous utilisons **RandomizedSearchCV** pour trouver les meilleurs paramètres :

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV

rf = RandomForestClassifier(class_weight='balanced', random_state=42,
    n_jobs=-1)

params = {
    'n_estimators': [100, 200],       # Nombre d'arbres
    'max_depth': [5, 10, 15],         # Profondeur max
    'min_samples_split': [5, 10],     # Echantillons min pour split
    'min_samples_leaf': [2, 5]        # Echantillons min par feuille
}

search = RandomizedSearchCV(
    rf, params, n_iter=10, cv=3,
    scoring='f1_macro', n_jobs=-1, random_state=42
)
search.fit(X_train_scaled, y_train_encoded)

```

```
best_model = search.best_estimator_
print(f"Meilleur Score F1: {search.best_score_:.4f}")
```

Listing 5.2 – Optimisation avec RandomizedSearchCV

Sauvegarde des artefacts

Les modèles entraînés sont sauvegardés dans `models/` :

```
import joblib

joblib.dump(best_model, 'models/rf_model.joblib')
joblib.dump(scaler, 'models/scaler.joblib')
joblib.dump(label_encoder, 'models/label_encoder.joblib')
joblib.dump(X.columns.tolist(), 'models/feature_names.joblib')
```

Listing 5.3 – Sauvegarde des modèles

Architecture du modèle

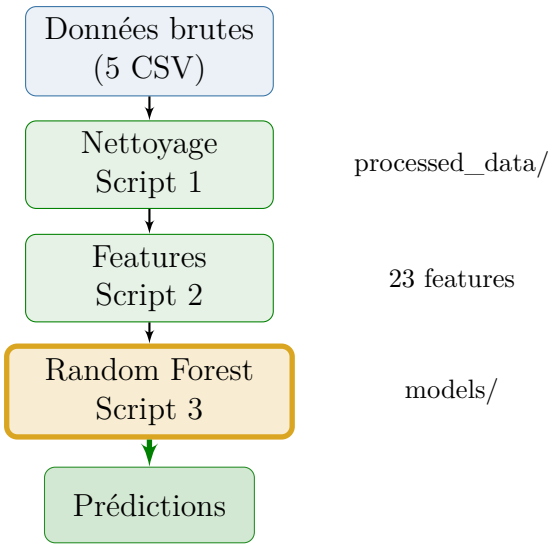


FIGURE 5.1 – Pipeline de Machine Learning

Simulation du tournoi

Équipes participantes

Les 16 équipes qualifiées pour la CAN 2025 au Maroc :

Équipe	Classement FIFA	Titres CAN
Maroc (Pays hôte)	13	1
Sénégal	17	1
Égypte	33	7
Côte d'Ivoire	39	3
Nigeria	44	3
Tunisie	47	1
Algérie	48	2
Cameroun	49	5
Mali	51	0
Afrique du Sud	57	1
RD Congo	61	2
Burkina Faso	75	0
Bénin	91	0
Tanzanie	105	0
Mozambique	108	0
Soudan	123	1

TABLE 6.1 – Équipes participantes à la CAN 2025

Fonction de prédiction

Le script `4_simulation_lib.py` charge le modèle et prédit les matchs :

```python
def predict_match(team1, team2):
    """Predit le vainqueur entre deux equipes"""
    X = get_live_features(team1, team2)
    X_scaled = scaler.transform(X)

    # Probabilites
    probs = model.predict_proba(X_scaled)[0]
    classes = label_encoder.classes_
    prob_dict = dict(zip(classes, probs))

    # Prediction
    pred_idx = np.argmax(probs)
    result = classes[pred_idx]

    if result == 'W': return team1, prob_dict
    elif result == 'L': return team2, prob_dict
    else: return 'Draw', prob_dict
```

Listing 6.1 – Prédiction d'un match

Simulation Monte Carlo

Nous simulons 1000 tournois complets :

```python
def simulate_tournament():
    bracket_16 = [
        ('Senegal', 'Soudan'), ('Mali', 'Tunisie'),
        ('Maroc', 'Tanzanie'), ('Afrique du Sud', 'Cameroun'),
        ('Egypte', 'Benin'), ('Nigeria', 'Mozambique'),
        ('Algerie', 'RD Congo'), ("Cote d'Ivoire", 'Burkina Faso')
    ]

    # Huitiemes de finale
    quarter_finalists = []
    for t1, t2 in bracket_16:
        winner, _ = predict_match(t1, t2)
        if winner == 'Draw':
            winner = t1 if FIFA_RANKING[t1] < FIFA_RANKING[t2] else t2
        quarter_finalists.append(winner)

    # Quarts, Demis, Finale...
    # (simulation complete)

    return champion
```

Listing 6.2 – Simulation du tournoi

Résultats et prédictions

Probabilités de victoire

Après 1000 simulations Monte Carlo, voici les probabilités :

Rang	Équipe	Champion	Finaliste	3ème place
accentgold !30 1	**Algérie**	**38.20%**	39.60%	12.50%
2	Égypte	13.50%	13.00%	19.30%
3	Côte d'Ivoire	12.90%	8.90%	16.20%
4	Sénégal	12.60%	13.70%	15.10%
5	Maroc	11.00%	11.20%	12.00%
6	Nigeria	5.20%	6.10%	8.50%
7	Cameroun	3.80%	4.20%	6.30%
8	Tunisie	2.10%	2.50%	4.80%

TABLE 7.1 – Top 8 des prédictions CAN 2025

Champion prédit

ALGÉRIE

Champion prédit avec 38.20% de probabilité

2 titres CAN • Classement FIFA : 48

Analyse des résultats

— **L'Algérie** domine nettement avec presque 40% de chances.

— Le **top 5** (Algérie, Égypte, Côte d'Ivoire, Sénégal, Maroc) représente 88% des chances cumulées.

— Le **Maroc**, pays hôte, bénéficie d'un avantage mais n'est pas favori.

— L'**Égypte** (7 titres CAN) reste un prétendant sérieux.

Application Web

Architecture technique

Nous avons développé une application web full-stack pour visualiser les prédictions :

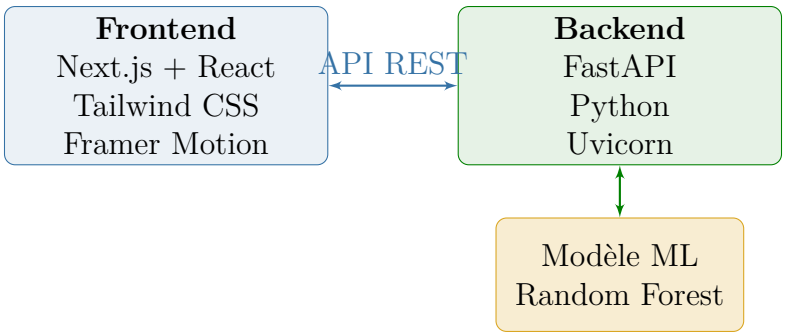


FIGURE 8.1 – Architecture de l'application web

Backend FastAPI

Le fichier `api.py` expose l'API de prédiction :

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class MatchRequest(BaseModel):
    team1: str
    team2: str

@app.post("/predict")
def predict_match_endpoint(req: MatchRequest):
    winner, probs = sim_lib.predict_match(req.team1, req.team2)

    return {
        "winner": winner,
        "team1WinProb": round(probs['W'] * 100, 1),
        "drawProb": round(probs['D'] * 100, 1),
        "team2WinProb": round(probs['L'] * 100, 1)
    }
```

Listing 8.1 – API FastAPI

Pages de l'application

L'application comprend 4 pages principales :

1. **Page d'accueil :** Affiche le champion prédit et le top 5.
2. **Page Match :** Simulation d'un match entre deux équipes.

3. **Page Bracket :** Tableau complet du tournoi.
4. **Page Simulation :** Simulation en temps réel.

Technologies utilisées

— **Next.js 14 :** Framework React moderne avec App Router.
— **Tailwind CSS :** Framework CSS utilitaire.
— **Framer Motion :** Animations fluides.
— **TypeScript :** Typage statique JavaScript.

Structure du projet

Arborescence des fichiers

```
CAN-2025-Predictions/
|-- 1_data_cleaning.py          # Nettoyage des donnees
|-- 2_feature_engineering.py    # Creation des features
|-- 3_train_model.py            # Entrainement du modele
|-- 4_simulation_lib.py         # Simulation du tournoi
|-- api.py                      # API FastAPI
|-- requirements.txt            # Dependances Python
|-- README.md                   # Documentation
|
|-- raw_data/                   # Donnees brutes (5 CSV)
|   |-- African national football from 2010-2024.csv
|   |-- Africa Cup of Nations Matches.csv
|   |-- fifa_ranking-2024-06-20.csv
|   |-- Champions.csv
|   |-- General Statistics For each Participated Team.csv
|
|-- processed_data/             # Donnees nettoyees
|   |-- cleaned_african_football.csv
|   |-- cleaned_can_matches.csv
|   |-- final_dataset_for_modeling.csv
|
|-- models/                     # Modeles entraines
|   |-- rf_model.joblib
|   |-- scaler.joblib
|   |-- label_encoder.joblib
|   |-- feature_names.joblib
|
|-- frontend/                   # Application Next.js
    |-- app/
    |   |-- page.tsx            # Page d'accueil
    |   |-- match/page.tsx      # Simulation match
    |   |-- bracket/page.tsx    # Tableau tournoi
    |   |-- simulation/page.tsx
    |-- components/
    |-- lib/data.ts             # Donnees des equipes
```

Listing 9.1 – Structure du projet

Dépendances

```
pandas
numpy
scikit-learn
joblib
matplotlib
seaborn
fastapi
uvicorn
```

Listing 9.2 – requirements.txt

Conclusion

Récapitulatif

Ce projet a permis de mettre en pratique les concepts fondamentaux du Machine Learning :

- **Data Engineering :** Nettoyage et préparation de 5 jeux de données.
- **Feature Engineering :** Création de 23 caractéristiques pertinentes.
- **Modélisation :** Entraînement d'un Random Forest avec optimisation.
- **Simulation :** 1000 simulations Monte Carlo du tournoi.
- **Déploiement :** Application web full-stack (FastAPI + Next.js).

Résultat principal

Notre modèle prédit l'**Algérie** comme champion de la CAN 2025 avec une probabilité de **38.20%**, suivie de l'Égypte (13.50%) et de la Côte d'Ivoire (12.90%).

Limites et améliorations

Limites actuelles :

- Dataset limité (500 matchs).
- Pas de prise en compte des blessures/suspensions.
- Facteurs humains non modélisables (moral, pression).

Améliorations futures :

- Intégrer des données de joueurs individuels.
- Utiliser des modèles plus complexes (XGBoost, Neural Networks).
- Ajouter des données en temps réel pendant le tournoi.

Compétences acquises

Ce projet nous a permis de développer des compétences en :

- Python et ses bibliothèques (Pandas, Scikit-learn).
- Pipeline de Machine Learning complet.
- Développement web full-stack.
- Travail en équipe et gestion de projet.

Merci pour votre attention.